

LAPORAN PRAKTIKUM

Provider & Database

Nama : Muhammad Djidzan Naufal Nugraha

NIM : 2311102189

Kelas : IF-11-03

A. Provider – State Management Flutter

1. Source Code Provider

a. main.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter_provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Counter with Provider',
      home: const CounterScreen(),
    );
  }
}

class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ini adalah aplikasi Counter'),
        backgroundColor: Colors.red,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('Isi Counter: '),
            Consumer<CounterProvider>(
              builder: (context, counter, child) {
                return Text('${counter.counter}');
              },
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    ),
  ),
  floatingActionButton: Column(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      FloatingActionButton(
        onPressed: () {
          Provider.of<CounterProvider>(context, listen:
false).increment();
        },
        child: const Icon(Icons.add),
      ),
      FloatingActionButton(
        onPressed: () {
          Provider.of<CounterProvider>(context, listen:
false).decrement();
        },
        child: const Icon(Icons.remove),
      ),
    ],
  ),
);
}
}

```

b. counter_provider.dart

```

import 'package:flutter/material.dart';

class CounterProvider with ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }

  void decrement() {
    if (_counter > 0) {
      _counter--;
      notifyListeners();
    }
  }
}

```

Konfigurasi Tambahan – pubspec.yaml

```

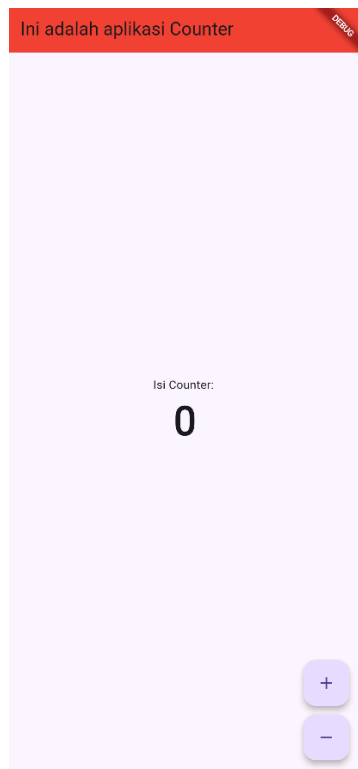
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.0.0

# Kemudian jalankan:
# flutter pub get

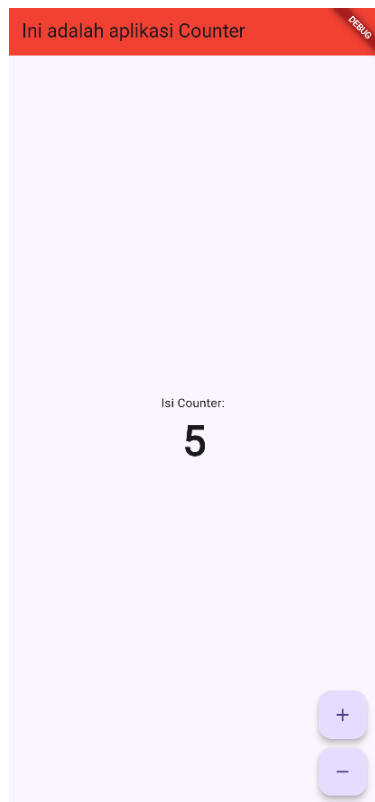
```

2. Screenshot Output Provider

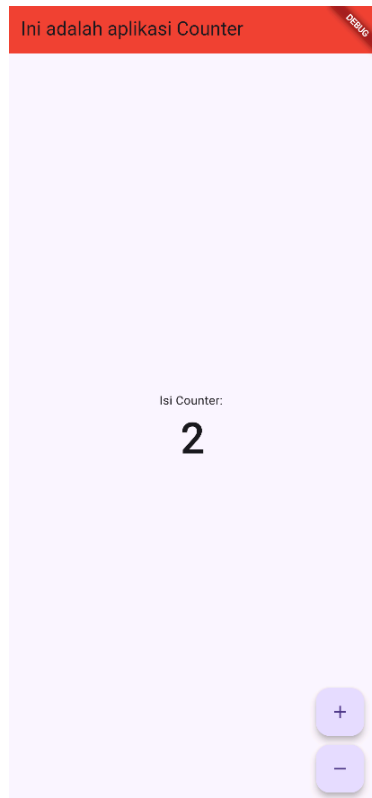
1. Tampilan awal aplikasi setelah berhasil dijalankan



2. Tampilan ketika nilai counter berhasil ditambah



3. Tampilan ketika nilai counter berhasil dikurangi



3. Penjelasan Program Provider

Program di atas adalah aplikasi Flutter sederhana yang mengimplementasikan state management menggunakan package Provider. Pada fungsi `main()`, seluruh aplikasi dibungkus oleh `ChangeNotifierProvider` yang bertugas menyuntikkan objek `CounterProvider` ke dalam widget tree. Kelas `CounterProvider` mewarisi mixin `ChangeNotifier` dan mengelola sebuah variabel privat `_counter` yang dapat diakses oleh widget lain melalui getter `counter`.

Kelas tersebut menyediakan dua metode utama, yaitu `increment()` untuk menambah nilai counter sebesar satu, dan `decrement()` untuk mengurangnya dengan kondisi nilai tidak boleh kurang dari nol. Setiap kali salah satu metode dipanggil, fungsi `notifyListeners()` dieksekusi agar seluruh widget yang bergantung pada data ini diperbarui secara otomatis.

Pada widget `CounterScreen`, tampilan utama aplikasi terdiri dari `AppBar`, sebuah teks keterangan, serta nilai counter yang ditampilkan menggunakan widget `Consumer<CounterProvider>`. Penggunaan `Consumer` memastikan hanya bagian tampilan yang membutuhkan data yang akan di-rebuild ketika state berubah, sehingga performa aplikasi tetap optimal. Dua `FloatingActionButton` yang ada di bagian bawah layar berfungsi memanggil metode `increment()` dan `decrement()` dari `CounterProvider` melalui `Provider.of<CounterProvider>(context, listen: false)`. Pendekatan ini memisahkan logika bisnis dari tampilan, menjadikan kode lebih terstruktur dan mudah dipelihara.

B. Database – SQLite & HTTP API

1. Source Code Database

a. main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'character_model.dart';
import 'db_helper.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SQLite & HTTP Demo',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}

class CharacterListScreen extends StatefulWidget {
  const CharacterListScreen({super.key});

  @override
  State<CharacterListScreen> createState() => _CharacterListScreenState();
}

class _CharacterListScreenState extends State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }

  Future<void> _loadFromLocal() async {
    setState(() => _isLoading = true);
    final localData = await DatabaseHelper.instance.getAllCharacters();
    setState(() {
      _characters = localData;
      _isLoading = false;
    });
  }

  Future<void> _deleteAllData() async {
    bool? confirm = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Hapus Semua Data?'),
        content: const Text('Apakah anda yakin?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Batal'),
          ),
        ],
      ),
    );
  }
}
```

```

        ),
        TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Hapus', style: TextStyle(color: Colors.red)),
        ),
    ],
),
);
if (confirm != true) return;
setState(() => _isLoading = true);
try {
    await DatabaseHelper.instance.clearTable();
    setState(() { _characters.clear(); });
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Semua data berhasil dihapus dari
sistem.')),
        );
    }
} catch (e) {
    if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('Terjadi kesalahan: $e')),
        );
    }
} finally {
    setState(() => _isLoading = false);
}
}

Future<void> _fetchFromAPI() async {
    setState(() => _isLoading = true);
    try {
        final response = await http.get(
            Uri.parse('https://jsonplaceholder.typicode.com/users'),
        );
        if (response.statusCode == 200) {
            final List<dynamic> dataJson = json.decode(response.body);
            await DatabaseHelper.instance.clearTable();
            List<Character> freshCharacters = [];
            for (var item in dataJson) {
                final char = Character.fromMap(item);
                freshCharacters.add(char);
                await DatabaseHelper.instance.insertCharacter(char);
            }
            setState(() => _characters = freshCharacters);
            if (mounted) {
                ScaffoldMessenger.of(context).showSnackBar(
                    const SnackBar(content: Text('Berhasil sync data dari
server!')),
                );
            }
        }
    } catch (e) {
        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(content: Text('Gagal konek internet, pastikan
online.')),
            );
        }
    } finally {
        setState(() => _isLoading = false);
    }
}

@override
Widget build(BuildContext context) {

```

```

return Scaffold(
  appBar: AppBar(
    title: const Text('Ensiklopedia NPC'),
    actions: [
      IconButton(
        icon: const Icon(Icons.delete_forever, color: Colors.redAccent),
        tooltip: 'Hapus Semua Data',
        onPressed: _characters.isEmpty ? null : _deleteAllData,
      ),
      IconButton(
        icon: const Icon(Icons.cloud_download),
        tooltip: 'Sync dari Server',
        onPressed: _fetchFromAPI,
      ),
    ],
  ),
  body: _isLoading
    ? const Center(child: CircularProgressIndicator())
    : _characters.isEmpty
    ? const Center(child: Text('Data kosong.'))
    : ListView.builder(
        itemCount: _characters.length,
        itemBuilder: (context, index) {
          final char = _characters[index];
          return ListTile(
            leading: CircleAvatar(
              backgroundColor: Colors.deepPurple,
              child: Text(char.id.toString()),
            ),
            title: Text(char.name),
            subtitle: Text('Username: ${char.username}'),
          );
        },
      ),
);
}

```

b. db_helper.dart

```

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import 'character_model.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('characters.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
    return await openDatabase(path, version: 1, onCreate: _createDB);
  }

  Future _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE characters (
        id INTEGER PRIMARY KEY,

```

```

        name TEXT NOT NULL,
        username TEXT NOT NULL
    )
    '');
}

Future<void> insertCharacter(Character character) async {
    final db = await instance.database;
    await db.insert(
        'characters',
        character.toMap(),
        conflictAlgorithm: ConflictAlgorithm.replace,
    );
}

Future<List<Character>> getAllCharacters() async {
    final db = await instance.database;
    final maps = await db.query('characters');
    return maps.map((map) => Character.fromMap(map)).toList();
}

Future<void> clearTable() async {
    final db = await instance.database;
    await db.delete('characters');
}
}

```

c. character_model.dart

```

class Character {
    final int id;
    final String name;
    final String username;

    Character({required this.id, required this.name, required this.username});

    factory Character.fromMap(Map<String, dynamic> map) {
        return Character(
            id: map['id'],
            name: map['name'],
            username: map['username'] ?? 'Unknown Username',
        );
    }

    Map<String, dynamic> toMap() {
        return {'id': id, 'name': name, 'username': username};
    }
}

```

Konfigurasi Tambahan – pubspec.yaml

```

dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.3.3
  path: ^1.9.0
  http: ^1.2.1

# Kemudian jalankan:
# flutter pub get

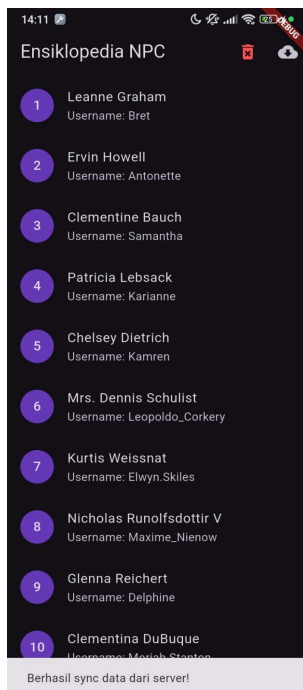
```

2. Screenshot Output Database

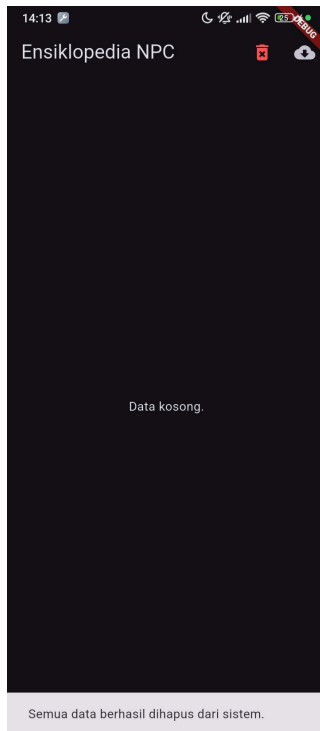
1. Tampilan awal saat data masih kosong (belum disinkronkan dengan server)



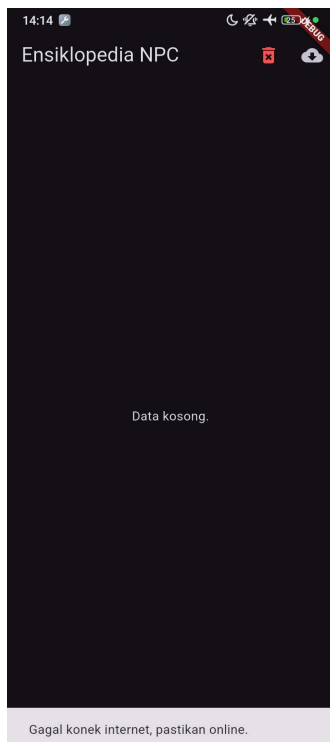
2. Tampilan setelah data berhasil disinkronkan dari jsonplaceholder.typicode.com/users beserta notifikasi 'Berhasil sync data dari server'



3. Tampilan pop-up konfirmasi hapus data dan notifikasi 'Semua data berhasil dihapus dari sistem' setelah penghapusan berhasil dilakukan



4. Tampilan ketika koneksi internet dimatikan atau URL server diubah, sehingga muncul notifikasi 'Gagal konek internet, pastikan online'



3. Penjelasan Program Database

Program di atas merupakan aplikasi Flutter yang mengintegrasikan dua teknologi utama, yaitu SQLite untuk penyimpanan data lokal dan HTTP untuk pengambilan data dari API eksternal. Saat aplikasi pertama kali dibuka, metode `initState()` secara otomatis memanggil `_loadFromLocal()` yang mengambil seluruh data karakter yang tersimpan di database lokal melalui `DatabaseHelper`, kemudian menampilkannya dalam bentuk daftar menggunakan `ListView`.

Pengguna dapat melakukan sinkronisasi data dengan menekan tombol ikon unduh pada `AppBar`, yang akan menjalankan metode `_fetchFromAPI()`. Metode ini mengirimkan permintaan HTTP GET ke endpoint `jsonplaceholder.typicode.com/users`, mengonversi respons JSON menjadi daftar objek `Character`, menghapus data lama yang tersimpan di SQLite, menyimpan data baru, lalu memperbarui tampilan. Jika koneksi gagal, aplikasi menampilkan pesan error kepada pengguna melalui `SnackBar`.

Fitur penghapusan data diimplementasikan pada metode `_deleteAllData()` yang menampilkan dialog konfirmasi sebelum benar-benar menghapus seluruh isi tabel. Kelas `DatabaseHelper` menggunakan pola Singleton untuk memastikan hanya ada satu instance koneksi database yang aktif sepanjang siklus hidup aplikasi. Kelas ini bertanggung jawab membuat file database `characters.db`, mendefinisikan skema tabel `characters`, serta menyediakan tiga operasi dasar: `insertCharacter()`, `getAllCharacters()`, dan `clearTable()`. Model `Character` mendefinisikan struktur data dengan atribut `id`, `name`, dan `username`, dilengkapi metode `fromMap()` dan `toMap()` untuk mempermudah konversi antara objek Dart dan format penyimpanan database.